

Holo — Technical Report

Assisted RGB capture ingestion and single-room reconstruction prototype Prepared 4 October 2026 · Companion to the [compliance matrix](#), [device matrix](#) and [runbook](#).

1. Scope and status

Holo is a local application and Python package that verifies phone capture exports, reconstructs single-room RGB geometry, and publishes a rough plan and interactive point cloud. It was prioritised around the hardware that was actually available — an assisted iPhone video export with intrinsics and ARKit poses — so the **video tier is the only end-to-end path**. Photos and raw LiDAR are ingested and validated but not reconstructed. Accuracy is **unverified** everywhere; a passing pipeline and a passing integrity audit are not a dimensional claim.

Area	State
Video ingest → preprocess → sparse → dense → surfaces → plan/ceiling → viewer	Implemented, audited (native Windows and Docker/Linux)
Photos (2–8/room)	Ingestion/validation only; reconstruction and stitching absent
LiDAR (depth + poses + intrinsics)	CLI raw ingestion only; conventions unverified; no fusion
Calibrated measurement intervals	Not implemented; no instrumented ground truth
Multi-room stitching, damage/scope	Not implemented (recorded gaps, §8)

Latest regression evidence: **187 Windows tests pass, Ruff clean**; integrity audits preserve original captures and every historical result.

2. Architecture

The system is a set of independently audited stages. Each stage reads an immutable upstream artifact, writes a fresh output, and verifies itself against source hashes before atomic publication, so a later failure never silently degrades to an earlier success.

upload → ingest+verify → preprocess+verify → fixed-pose sparse+audit → CUDA RGB stereo+audit → surfaces+audit → floor/wall/ceiling estimate → atomic viewer publication → plan + interactive 3D

Stage	Owner	Key output	Independent verification
Ingestion	src/cozmo_ingestion	canonical-capture-1/2 bundle	Source hash, schema, clock/frame association
Preprocessing	src/cozmo_preprocessing	selected views, exact K/poses, IMU	Hash, grid, K/pose/IMU values, pair identities
Sparse	src/cozmo_reconstruction	fixed-camera model, accepted tracks	Camera movement, feature lineage, atomic publish
Dense	.dense subpackage	source-world cloud + masks	Recomputes masks/cloud/report; audits model cameras
Surfaces	.surfaces subpackage	12 plane candidates + RGB evidence	Recomputes planes/labels/patches
Plan/ceiling	.viewer	SVG/JSON plan, rigid display cloud	Full upstream chain recheck, 8 bound assets

Design rules. Typed immutable request/policy objects; ports and injected backends (ports.py); pure geometry functions separated from lifecycle; no HTTP import in geometry code; HTTP only admits/reads allowlisted, hash-checked files. Runtimes: **native Windows** (CPU PyCOLMAP + official COLMAP 4.2.1 CUDA executable), **Docker** (CPU sparse; GPU overlay for dense). auto/dense/preview modes are explicit; required dense fails visibly rather than downgrading. Jobs are isolated, serialised, bounded (3,600 s automatic; 1,800 s dense; single-instance storage) and survive restart as failed-not-lost runs.

3. Tier design and device matrix

Tier	Input contract	Implemented output	Boundary
Assisted video	Sensor Recorder ARKit RGB, per-frame K, camera-to-world poses, IMU	Full dense pipeline, rough plan/ceiling, 3D	Working development path; other exporters NOT_RUN
Photos	JPEG/PNG per-room folders, no K/poses/depth	Canonical bundle + EXIF evidence	Stops after ingestion

Tier	Input contract	Implemented output	Boundary
LiDAR	uint16 depth + confidence, K/poses	Raw-data CLI bundle (CONVENTIONS_UNVERIFIED)	Ingestion only; no fusion
Plain MP4	No K/poses	Rejected explicitly	NOT_IMPLEMENTED
Android	New adapter required	Deferred	No claim

Hardware actually exercised: **tested phone** iPhone18,3, iOS 26.6.1, app 1.5/build 5; photo fixture was a Galaxy S24 (not iPhone). **Processing machine** Windows 11 (10.0.26200), ASUS TUF A14, Ryzen AI 9 HX 370 (12C/24T, ~31.1 GiB usable), RTX 4060 Laptop (driver 610.74), PyCOLMAP 4.2.1 CPU + COLMAP 4.2.1 CUDA executable. The ~415 MB CUDA archive is fetched by a checksum-pinned script, not vendored; no candidate-owned infrastructure is required.

4. Drift handling

Holo intentionally **does not run VIO, pose optimisation or loop closure**. It preserves the source ARKit trajectory as the single metric gauge:

- Ingestion checks clock/frame association (Sensor Recorder constant offset; Stray affine fit, slope $\leq 1\%$, residual ≤ 10 ms) and records tracking-state and pose-speed findings without repairing geometry.
- Preprocessing keeps exact source K/poses, flags apparent source-pose speeds above 3 m/s and low-baseline (< 2 cm) links, and reports supplied-pose Sampson residuals as **consistency diagnostics, not a correction**. The double-room export records one 3.60 m/s step at 40.548 s; the single-room chain's largest selected gap is 0.5002 s.
- Sparse reconstruction fixes cameras/rigs, disables intrinsic and sensor refinement, snapshots cameras before/after, and blocks publication on unexpected camera movement.
- Dense stereo reuses the same fixed poses; IMU remains diagnostic only.

Consequence: **drift correction with an on/off stitched-footprint ablation is FAIL as specified**. Without a surveyed repeat capture or ground truth, residual drift cannot be separated from pose error, so no correction is applied rather than an unvalidated one.

5. Error budget

There is **no calibrated error budget**, because no instrumented truth exists. What is recorded is a chain of bounded engineering thresholds and observed residuals:

Quantity	Value / threshold	Type
Sparse median / p90 residual (100-view)	1.21 / 2.76 px	Observed, internal
Sparse acceptance gate	≤ 4 px every observation, $\geq 1.5^\circ$ ray angle	Engineering choice
Dense depth agreement	$\leq 2\%$ relative, ≤ 1 px reprojection, $\geq 1^\circ$ ray, 2 neighbours	Engineering choice
Dense Z validity	0.1–20 source estimated m	Engineering choice
Dense accepted pixels (100-view)	24.3172% of selected pixels	Coverage, not room completeness
Ceiling estimate	≈ 2.573 m (upper-envelope heuristic)	Provisional, not a confidence interval
Opening width	assumed illustrative 0.8 m	Assumption, not measured
Ceiling gate (≤ 1.5 cm)	NOT_RUN	No instrumented truth
Repeat per-wall (≤ 1 cm / 0.5%)	NOT_RUN	Identical-cache repeats are not separate acquisitions

Error enters at: unresolved phone intrinsics/pixel-centre convention (§6), fixed ARKit pose drift (§4), low-texture/weak-parallax rejection, furniture/curtain contamination in plane fitting, and heuristic floor/rectangle/ceiling selection. The single-room reduced support (24% dense acceptance) is dominated by plain walls, floor, ceiling and transitions, so the plan is a **provisional hypothesis** that occlusion can bias small and furniture can bias large. Integration/clock issues are bounded but not tied to metric ground truth.

6. Calibration analysis

A user-declared A4 reference (0.210×0.297 m, thickness $0 \leq t < 0.010$ m) was used as an **independent check**, never as a scale input. Six reviewed corner marks were triangulated against the fixed source cameras with general IPPE:

Check	Result	Outcome
Closest-ray triangulation	max 10.1415 px (corner 2, rank 165)	Rejected (gate 4 px)

Check	Result	Outcome
Per-corner RMS	2.9058 / 0.5294 / 6.6984 / 2.1628 px	Inconsistent
±0.5 px principal-point shifts	max 10.1370 / 10.1461 px	Both rejected
Leave-one-view-out	all six subsets rejected, 9.1260–11.3326 px	Not a single bad view
Pixel-objective point fit	RMS 3.7940 px, max 10.6775 px	No correction adopted
Per-view LM pose refinement	improves RMS (e.g. 18.64 → 3.54 px) but 3 views still fail	Ambiguity persists

Interpretation. The failure is not explained by ARKit inconsistency alone (per-view rectangle residuals do not use world poses), nor by half-pixel convention or one view. Under the current rectangle/pinhole assumptions it remains unattributed between projection/calibration, object shape, correspondence uncertainty and source-pose error. Because the gate fails, **no scale, floor-height or intrinsic correction is adopted** and floor comparison is withheld. Resolving calibration needs an independent measured rigid target with stated uncertainty; re-optimising the same four corners cannot supply it. Source distortion and exact pixel-centre convention remain **UNVERIFIED**.

Independent checks reinforce the "rough, not certified" stance: planar surface fitting proposes 12 candidates (8 multi-view, 4 weak) with 480,423/878,459 voxels assigned, but multi-view support never confirms architectural identity (P01 follows the **bed**, not the floor). Region-reviewed boundary work retains only 11 local floor cells and one ≈0.599 m wall-patch projection — no certified corner, junction or closed polygon.

7. The fix loop

A real regression and its repair show the loop end to end.

1. **Detect.** The new native CUDA dense run (ae77c443...) completed and passed integrity checks but inferred a **4.539 × 4.055 m / 18.405 m²**, 2.551 m-ceiling plan versus the earlier ≈3.327 × 3.851 m Docker result for the same room.
2. **Baseline.** The failing result, viewer, cloud/colours/floor transform and historical jobs were frozen and committed as error baseline **d9723e2** (pushed to `main`). Evidence: [regression evidence](#).
3. **Diagnose.** Root cause was in the estimator, not the backend: `dense_automatic.dense_room` fed **raw projected candidate spans** (every occupied part of a vertical plane, without required vertical coverage at each endpoint) into rectangular completion, so extra low-height extensions could set the outer rectangle. Native plane fitting differed from Docker, exposing the sensitivity.
4. **Fix (b789117).** Fit the existing height-persistent `suggested_spans` (≥2 supported height bands, ≥0.8 m vertical extent, contiguous intervals) instead; fail visibly with `DENSE_ROOM_SUPPORT_INSUFFICIENT` rather than falling back to the oversized envelope.
5. **Verify.** An audited **CPU re-publication of the same 848,220-point cloud** gives **3.147 × 3.706 m / 11.662 m² / 2.573 m** with **no re-run of stereo**. A synthetic floor/four-wall test where a 2 m low-height extension enlarges raw completion by >30% now leaves the filtered room unchanged; a companion test checks visible failure on missing support. Source/viewer hashes are preserved and the failing and corrected outlines coexist.

This is a **retrospective** fix loop: the formal declaration and a worst-measured-gate comparison were not produced in advance, which the compliance matrix records honestly.

8. Known failure modes and gaps

Mode	Symptom	Current behaviour
Occluded/short wall ends	Plan underestimates size	Fail-visible if persistent support is missing
Tall furniture / curtains	Qualify as vertical or floor surfaces	Flagged as unconfirmed; manual RGB review required
Low-texture walls, floor, ceiling, edges	Sparse/dense rejection, scattered outliers	Coverage left open; no forced closure
Weak parallax / few neighbours	Views dropped from stereo	Recorded in <code>dense-selection.json</code>
Mirrors / glass / wet-look / low light	Unsupported, not benchmarked	No condition-labelled challenge set (NOT_RUN)
Gaussian novel views	Blur/tear; not structural	Retained but deferred
Plain MP4 / Android / HEIC	Unsupported format	Explicit rejection with actionable message
Long captures	In-memory records/features	Limits: 36,000 frames, 2,400 candidates

Recorded gaps (no output manufactured): surface damage classification and metric extent; concealed-damage flags with a fired rule; scope line items keyed to surfaces; calibrated measurement intervals; multi-room placement/adjacency and whole-property stitching; JSON to the evaluator's published schema (not supplied); LiDAR fusion and photo scale/reconstruction.

9. Limitations

Delivered accuracy is comparable to a coarse site sketch, not a survey. The rough rectangle, doorway and ceiling are hypotheses on an **uncalibrated source-scale gauge** with fixed, uncorrected poses. Tests and audits certify behaviour, lineage and integrity — not dimensions or complete rooms. The evaluator's Round-1 contract and output schema were not available, so no scored worst-gate claim is made. Hardware exposure was limited to one non-LiDAR iPhone and one Windows/NVIDIA machine; cross-device, cross-platform and clean-machine timing remain unverified.